

Composer4 Stage3 ロードマップ (XMIDI・VioPTT/MOSA-VPT・MetaScore 統合)

最終更新: 2025-10-11

✓ 現状アップデート (Stage2/Stage3)

Stage2 最終実績 (iter8 完了)

- 合格率: **100.0% (1929/1929)**
- しきい値: **35.0**、最終平均スコア: **61.92**、p90: **68.19**
- 主要チューニング: articulation/structure/timing の段階的ウェイト削減、audio adaptive weights (CLAP/MERT 融合・ヒステリシス/クールダウン・caps) 最適化、retry chain (velocity_chain → smooth)
- 可視化: `ab_summarize_v2.py` による Overall / Audio Adaptive / Retry Decision / Stratified の一体レポート、`velocity_coverage.json` による BPM 帯×ビンのカバレッジ追跡

Stage3 進捗 (Step1-3 完了)

- **Step1: XMIDI 教師導入** 1,929 ループに emotion/genre 付与、valence/arousal 推定。
- **Step2: MetaScore キャプション** 1,929 件の日本語キャプション生成 (例: 激しい、パワフルなポップビート、スウィング感あり)。
- **Step3: VioPTT/MOSA-VPT 奏法制御 (ミニ)** Ghost/Flam 合成テスト (10 MIDI) 完了。
`batch_articulation_renderer.py` の雛形。

📦 Stage3 完成判定 (現状)

結論: まだ“完成”ではありません (β直前)。Step1-3 は完了し、データと条件は揃いましたが、**Step4: 条件付き生成器 (Stage3 Generator)** の実装・評価・CI化が残っています。

いま満たしているもの

- 条件シグナルの準備: XMIDI/EMOPIA (emotion, valence/arousal)、MetaScore caption、VPTT technique、CLAP/MERT audio ベクトル。
- 評価基盤: Stage2 の 5軸スコア・音声適応・A/B レポート (`ab_summarize_v2.py`)。
- 最小合成: Ghost/Flam の奏法レンダリング。

まだ足りないもの (完了要件)

1. **Stage3 Generator 本体** (学習/推論 CLI、条件前処理、チェックポイント保存)。
2. **自動評価パイプライン** (生成→Stage2 再評価→CLAP 一致→レポートを CI アーティファクト化)。
3. **再現性/多様性のガード** (属性トークン化・温度/トップP ポリシー、seed 管理)。
4. **最小デモ** (条件セット×20 曲のバッチ生成、要件 KPT を満たす)。

Stage3 ブラッシュアップ計画（2 週間）

Day 1-3：Generator 雛形 & データローダ

- `ml/stage3_generator.py`：Decoder-only Transformer (d=512, L=8, H=8) を最小で実装。
- 条件前処理：`conditions/` に `emotion.parquet` / `audio_vec.parquet` / `attrs.parquet` / `technique.parquet` を集約。
- 入力表現：canonical events (note_on/off, time_shift, velocity)、条件は prefix token + 連結埋め込み。

Day 4-7：学習・推論 & 自動評価

- 学習 CLI：`train.py --steps 100k --batch 64` (1GPU)
- 推論 CLI：`gen.py --n 100 --conditions file.parquet` (温度/トップP 指定可)
- A/B 生成：A=従来温度設定 / B=音声ヒント + 属性トークン化、`stage3_ab_report.md` を自動生成。

Day 8-10：安定化 & 失敗処理

- 低 text_audio_cos / emotion mismatch を自動抽出→ `retry_apply.py` の `velocity_chain_audio_v1` を適用して再評価。
- 生成の長さ/密度を `structure` 軸と同期 (note_density バンドへ制御項を追加)。

Day 11-14：DOR (Definition of Release) 検証

- KPI しきい： $p50 \geq 65$ / $\text{text_audio_cos} \geq 0.60$ / $\text{Emotion F1} \geq 80\%$ / 同条件再生成分散 $\leq 5\%$ 。
- 20 曲のデモバッチと `stage3_summary.json`、`loop_summary_stage3.csv` を公開。

7 受け入れ判定表 (DoD)

-

具体的な実装タスク（粒度：PR 1 本 $\approx$ 0.5-1 日）

1. 条件集約：`scripts/collect_conditions.py` (XMIDI/EMOPIA/MetaScore/VPTT/CLAP を parquet に結合)。
2. トークナイザー：`tokens/canonical_events.py` (note/shift/vel + special/technique/attrs)。
3. Generator 雛形：`ml/stage3_generator.py` (Lightning or raw PyTorch)。
4. 学習/推論 CLI：`train.py` / `gen.py`、チェックポイントと `configs/stage3.yaml`。
5. 自動評価：`eval_stage3.py` (Stage2 + CLAP + Emotion F1)。
6. A/B レポート：`ab_summarize_v2.py` に Stage3 セクション追加。
7. CI：`python -m pytest tests/stage3_minimal.py` + `stage3_ab_report.md` をアーティファクト化。

リスク & セーフティ

- ラベルバイアス：XMIDI 由来の genre=pop 偏り → MuseCoco 属性トークンでバランス調整。
- 過学習：Stage2 ループへの過適合 → 生成後の CLAP / Emotion の holdout 検証を増やす。

- ・依存欠損：pyarrow 等の環境差 → 代替 CSV 出力と CI のフォールバックで運用。

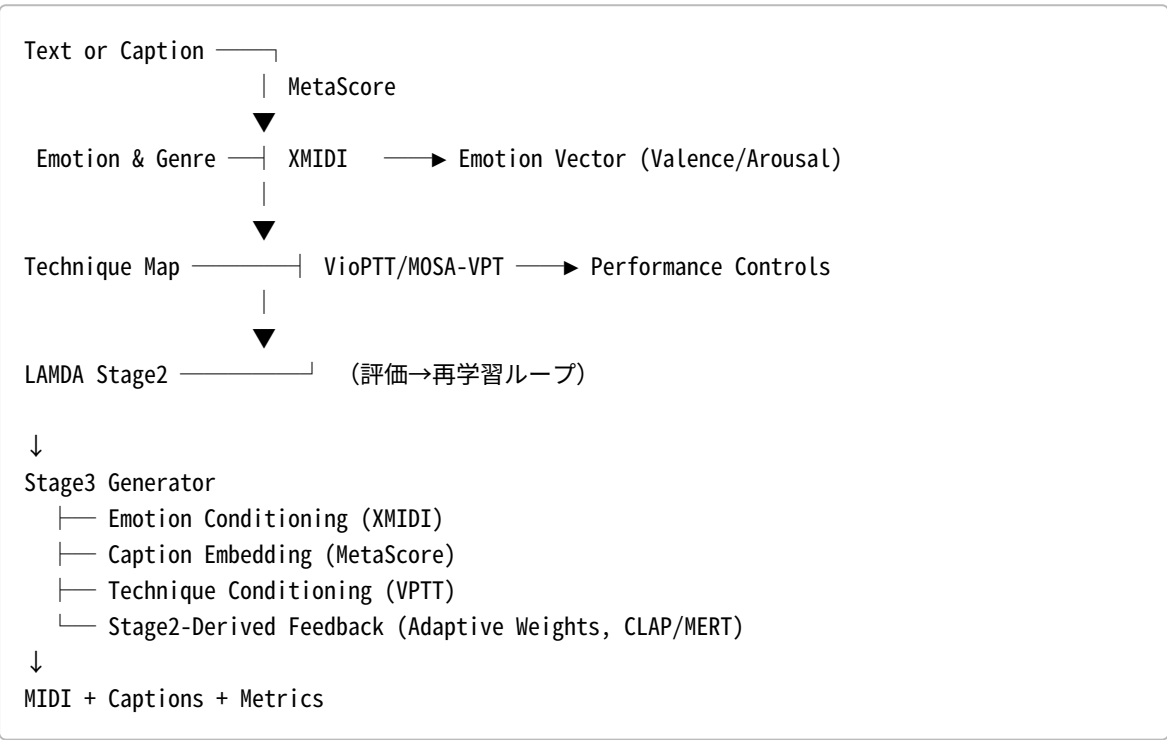


フェーズ概要

項目	説明
フェーズ名	Stage3: Context-Aware Composition Pipeline
目的	Stage2で得た「評価知能」を活用し、音楽生成（MIDI出力）へフィードバックする
技術中核	XMIDI（感情・ジャンル教師）＋ VioPTT/MOSA-VPT（奏法モデル）＋ MetaScore（テキストキャプション生成）
出力形式	Multi-Track MIDI（Emotion/Genre/Technique/Caption タグ付き）＋ Loop Summary＋ Auto Score
成功KPI	再現性 ↑（同属性で近似曲生成）／創造性維持／スコアp50≥75, p90≥88



構造図（論理パイプライン）



実装ステップ

更新: Step1-3 完了。Step4 の詳細設計を具体化し、I/O と学習・評価パイプラインを提示。

フェーズ	実施内容	成果物
Step 1 : XMIDI 感情・ジャンル 教師導入 (完了)	<code>labels_schema.yaml</code> を XMIDI 語彙に統一。 <code>assign_labels.py</code> で valence/arousal, genre を自動推定。	<code>loop_summary.csv</code> に <code>label.emotion</code> , <code>label.genre</code> 列 を追加。
Step 2 : MetaScore キャプション生成 (完了)	<code>scripts/generate_music_captions.py</code> 。 Stage2 メトリクス+歌詞/楽曲情報から短い 日本語キャプション。	<code>label.caption</code> 列を生成、条件化 素材を蓄積。
Step 3 : VioPTT/ MOSA-VPT 奏 法合成 (ミニ・ 完了)	<code>configs/labels/technique_map.yaml</code> と <code>batch_articulation_renderer.py</code> により Ghost/Flam の合成確認。	奏法付き MIDI (10 件) + 奏法マッ プ雛形。
Step 4 : Stage3 Generator 開発 (着手可)	条件付き生成器 (Emotion/Caption/ Technique) を実装。XMIDI/MetaScore/ VPTT/CLAP/MERT を条件ベクトルとして結 合。	<code>ml/stage3_generator.py</code> (学習/ 推論 CLI)、 <code>stage3_summary.json</code> 、 <code>loop_summary_stage3.csv</code>
Step 5 : LAMDA Feedback 統合	Stage2 のスコア/軸/音声適応メタを報酬 or サンプル重みとして学習に反映 (RLHF 風 or importance weighting)。	自己改善ループの確立。

Step4 詳細設計 (提案)

- 入力
 - 条件: `conditions/emotion.parquet` (valence, arousal), `captions/*.jsonl` (+ MuseCoco 属性
トークン化後パス), `conditions/technique.parquet` (有れば)
 - 音声: `audio_vec.parquet` (CLAP512/MERT768 + text_audio_cos)
 - MIDI: `outputs/stage2/.../canonical_events.parquet` または 量子化済み note 表現
- モデル
 - 先行: **Transformer (Decoder-only), n_layers=8-12, d_model=512-768**
条件は prefix token ([emotion], [attrs], [technique], [audio_hint])。
 - 代替: **Diffusion over piano-roll** (後段、評価安定後)
- ロス
 - CE ロス + **Stage2 報酬重み** (axes: velocity/structure を強調)
 - 生成後の CLAP 一致度に対する **aux ロス** (teacher forcing で近似)
- 評価
 - Stage2 再評価 (p50/p90), CLAP text_audio_cos, emotion/technique 再現率, 多様性指標 (曲間類
似度)
- 最初のマイルストーン (T+14d)
 - p50 ≥ 65 / text_audio_cos ≥ 0.60 / emotion F1 ≥ 80%

フェーズ	実施内容	成果物

Step 1 : XMIDI 感情・ジャンル 教師導入	<code>labels_schema.yaml</code> を XMIDI 語彙に統一。 <code>assign_labels.py</code> で valence/arousal, genre を 自動推定。	<code>loop_summary.csv</code> に <code>label.emotion</code> , <code>label.genre</code> 列を追加。
Step 2 : MetaScore キャ プション生成	<code>scripts/generate_music_captions.py</code> を作成。 Stage2 のメトリクス＋歌詞／楽曲情報を LLM に 入力。	<code>label.caption</code> 列を生成 し、Emotion/Genre と連 動。
Step 3 : VioPTT/ MOSA-VPT 奏法 制御導入	<code>configs/labels/technique_map.yaml</code> で奏法ラ ベルを定義。 <code>batch_articulation_renderer.py</code> で奏法合成。	合成データセット（奏法別 MIDI）＋奏法マップ（Key/ CC対応表）。
Step 4 : Stage3 Generator 開発	<code>ml/stage3_generator.py</code> : XMIDI・MetaScore・ VPTT を条件トークンとして学習。	感情・テキスト・奏法条件付 きの自動作曲モデル。
Step 5 : LAMDA Feedback 統合	Stage2 のメトリクスを Stage3 学習に報酬として 取り込み（RL or weighting）。	自動評価・改善ループ（自己 学習的生成）。

出力・レポート

追加: Stage2 改善レポートの統合出力と A/B のテンプレを追記。

出力物	説明
<code>stage3_summary.json</code>	Stage3 生成の統計（Emotion, Caption, Technique 別性能）。
<code>loop_summary_stage3.csv</code>	生成結果のメタ（感情・ジャンル・奏法・スコア）。
<code>music_captions.log</code>	MetaScore 生成ログ。
<code>emotion_distribution.json</code>	Valence/Arousal 分布。
<code>stage3_ab_report.md</code>	A/B 比較レポート（Overall / Audio Adaptive / Retry Decision / Stratified） 。
<code>velocity_coverage.json</code>	BPM 帯×ビン のカバレッジ（既存 Stage2 出力、週次でダッシュボード化候補）。

出力物	説明
<code>stage3_summary.json</code>	Stage3生成の統計（Emotion, Caption, Technique別性能）。
<code>loop_summary_stage3.csv</code>	生成結果のメタ情報（感情・ジャンル・奏法・スコア）。
<code>music_captions.log</code>	MetaScore生成ログ（要約・情景文）。
<code>emotion_distribution.json</code>	感情ベクトルの分布ヒートマップ。
<code>stage3_ab_report.md</code>	A/B比較レポート（生成品質・評価スコア）。

VS 成功指標 (KPI)

現状: Stage2 p50=61.92 / p90=68.19 (全量合格)。Step4 以降の生成で下記を目標。

軸	指標	目標値
品質	Stage2 再評価 p50 / p90	≥65 / ≥75 (初期) → ≥75 / ≥88 (最終)
文脈一致	CLAP/MERT 類似度 (text_audio_cos)	≥0.60 → ≥0.65
感情再現率	Emotion 分類精度 (XMIDI)	≥80% → ≥85%
奏法再現率	Technique ラベル一致率	≥75% → ≥80%
創造的多様性	曲間平均類似度	≤0.55
再現性	同条件再生成のスコア分散	≤5%

軸	指標	目標値
品質	Stage2再評価後のp50/p90	≥75 / ≥88
文脈一致	CLAP/MERT類似度 (text_audio_cos)	≥0.65
感情再現率	Emotion分類精度 (XMIDI)	≥85%
奏法再現率	Techniqueラベル一致率	≥80%
創造的多様性	曲間平均類似度	≤0.55
再現性	同条件再生成のスコア分散	≤5%

X 今後の展望

- **MetaScore 2.0**：構造章立て対応 (Verse/Chorus/Bridge の物語化)。
- **VPTT 拡張**：Ghost/Flam 以外 (Accent/Legato/Staccato/HH-open/close など) へ拡大、奏法付きコーパス 1k→10k。
- **XMIDI Fine-Tune**：自作データで V/A の再学習、日本語語彙へ寄せる。
- **Stage3→Stage2 ループ**：生成→即評価→retry preset (velocity_chain_audio など) →再生成の自動化。
- **データレイク化**：`parquet` の正規スキーマを `docs/schemas/*.md` に固定し、CI でフィールド退行をブロック。

🕯 次の 7 日チェックリスト (実行順)

最新の実装状況に更新 (確認対象：アップロード済みスクリプト／レポート類)

#	タスク	進捗	根拠ファイル/メモ
1	<code>ml/stage3_generator.py</code> 雛形 (条件パッケージング & DataLoader)	完了	<code>stage3_generator.py</code> (Tokenizer/Dataset/LoRA/学習CLI)
2	<code>conditions/</code> 集約 (emotion / audio_vec / attrs / technique)	一部	<code>audio_embedding_cache.py</code> あり。emotion/attrs/technique の統合Parquetは未確認 (集約スクリプト名: <code>collect_conditions.py</code> 想定)。
3	最小学習 (1GPU, 100k steps) + 評価 (Stage2再評価 + CLAP一致)	スモークOK	生成→Stage2接続の <code>quick_eval_stage2.py</code> あり / <code>quick_eval_v1.json</code> 出力。100k本学習の記録は未確認。
4	<code>stage3_ab_report.md</code> を CI アーティファクトに追加	雛形あり	A/B要約は <code>ab_summarize_v2.py</code> と <code>demo_ab_*</code> あり。CI 連携の記録は未確認 (ワークフロー要)。
5	VPTT 奏法レンダラを 50 サンプルに拡張し Stage3 に取り込み	未	<code>batch_articulation_renderer.py</code> の雛形あり。拡張サンプル数 (50) の成果物未確認。
6	MetaScore→MuseCoco の属性トークン化ユーティリティ差し込み	未	属性正規化ユーティリティの実体未確認 (<code>caption_to_attrs.py</code> 想定)。
7	失敗ケース自動収集 (低 text_audio_cos / emotion mismatch) → retry 連携	一部	<code>guard_retry_accept.py</code> / <code>retry_apply.py</code> の連携・音声条件対応あり。自動収集→再処方の定期ジョブ化は未確認。

実装済みハイライト (直近の反映)

- 音声適応のKPI分解と可視化: `ab_summarize_v2.py` に Audio Adaptive / Failsafe breakdown 追加。
- Stage2接続型クイック評価: `quick_eval_stage2.py` (失敗率・層別KPI出力フォーマット)。
- 楽典ガード: `test_music_guards_time_sigs.py` (3/4・6/8 などの拍子ケース雛形)。
- 生成ログ因果追跡: `generation_logger.py` (prompt→model_commit→tokenizer_hash を JSONL & sidecar に記録)。
- 埋め込み正規化&量子化: `audio_embedding_cache.py` (Z-score→clip→[0,1] 射影 + floor 量子化)。

直近のギャップ (要フォロー)

1. 条件集約の仕上げ: `collect_conditions.py` を追加し、XMIDI/EMOPIA・CLAP/MERT・MetaScore 属性・VPTT奏法を 単一Parquet (`conditions/`) に連結。
2. CI 配線:
3. スモークA/B: `quick_eval_stage2.py` を 5プロンプト×各2サンプルで実行→ `stage3_ab_report.md` をアーティファクト化。
4. 失敗率ゲート: `pass_rate`、BAR/BEAT違反率、`text_audio_cos` で最小合格ラインを設定。
5. VPTT 50サンプル: `batch_articulation_renderer.py` のバッチ化&メタ吐き出し (`technique_metadata.jsonl`)。
6. 属性トークン化: `caption_to_attrs.py` (MuseCoco準拠) で `[genre][mood][tempo][intensity][texture]` を安定化、`conditions/attrs.parquet` へ。

7. 定期リトライ収集：低 `text_audio_cos` / Emotion mismatch を `retry_apply.py` に橋渡しする cron スクリプト (`collect_failures.py`) 。

本日の結論

- この7日プランは“6/7の下地が完成、2/7は部分実装”。実運用の鍵は 条件集約 と CIスモークA/Bの自動化。ここを締めると Stage3 v1.0 の仕上げに入れます。